Introduction

There's a specific kind of bug that only shows up after your workflow has already earned your trust. You build an n8n flow that reads a CSV, transforms it, and writes out a spreadsheet. You test it, it works, you ship it. Then weeks later, someone uploads a real file instead of a test file, and the whole thing grinds to a halt or crashes outright.

This is one of those bugs that looks like it could be anything — a bad node, a timeout, a flaky server — but it almost always comes down to one setting nobody thinks to touch: how n8n stores binary data while a workflow runs.

This matters beyond n8n specifically. It's a core systems concept that shows up in every data pipeline you'll ever build: does this tool stream data, or does it load the whole thing into memory first? Understanding the difference is what separates "works on my test data" from "works in production," and it's exactly the kind of judgment that separates someone who glues nodes together from someone who actually understands what's running underneath them.

Problem Overview

Here's the situation in plain terms: you have an n8n workflow that reads a CSV file, does some processing, and writes the result back out as a spreadsheet. It works perfectly in testing. In production, the files are much bigger — tens of thousands of rows instead of a few hundred — and the workflow crashes or hangs.

The question isn't "what node is broken." Nothing is broken. The question is: where does the actual file content live while the workflow is executing, and what happens to that location as the file gets bigger?

In n8n, every item that flows between nodes can carry two things: a JSON payload (structured data you can read and manipulate) and an optional binary property (the raw file itself — your CSV, your spreadsheet, your image). By default, that binary property lives fully in memory for the entire execution. A small file sits there without issue. A large file means n8n is holding the entire thing in RAM simultaneously while every single node in the chain touches it. Eventually, you run out of memory, and the workflow dies.

Example

Imagine two runs of the exact same workflow:

Run 1: A CSV with 500 rows, maybe 50KB. `Read Binary File` loads it, `Extract From File` parses it into JSON, you map a couple of fields, `Convert to File` turns it back into a file, and it writes to disk. Total memory footprint: trivial. Runs in under a second.

Run 2: The same workflow, same nodes, same logic — but the CSV now has 50,000 rows and is several hundred megabytes. n8n still treats the file the same way by default: it loads the entire blob into memory, and keeps it there as it passes from node to node. At some point, the process either slows to a crawl under memory pressure or gets killed outright.

Nothing in your workflow logic changed between these two runs. Only the file size changed. That's the entire bug.

Intuition

The mental model that unlocks this is simple: think of an n8n item like an email. The JSON is the body — plain, structured text you can read directly. The binary property is the attachment — the actual file riding along with the item.

Here's the part that trips people up: it's natural to assume that attachment is just a *reference* — like a file path pointing somewhere on disk, the way a lot of systems handle "big" data. It isn't, by default. It's the literal bytes of the file, attached directly to the item object, traveling through every node in the workflow as that item moves along.

That distinction — reference versus actual payload — is the whole story. A path is cheap to pass around no matter how big the file behind it is. A fully loaded blob is not; its cost scales directly with file size. So the moment your file size crosses from "trivial" to "substantial," you've crossed from "cheap to pass around" to "expensive to hold in memory," and nothing about your workflow's logic needs to change for that to become a crash.

An experienced engineer doesn't discover this by reading n8n's internals — they discover it by asking the right question the moment something that worked on test data breaks on real data: "what's actually different about the data, and what does the system do differently because of it?" Size is the different thing here. Memory footprint is the "what the system does differently."

Brute Force Solution

The "brute force" approach, in a sense, is what most workflows do by default — and it's not a solution so much as the thing that breaks:

Idea: Load the entire binary file into memory for the full lifetime of the workflow execution, and pass that fully-loaded blob from node to node.

Algorithm: 1. `Read Binary File` (or an equivalent trigger) loads the file's full contents into memory and attaches it as the binary property. 2. Every subsequent node receives the item with that binary payload still attached, in memory. 3. `Extract From File` parses the whole thing at once. 4. Any transformation happens with everything resident in RAM simultaneously.

Advantages: - Dead simple — no configuration needed, it's the default behavior. - Fast and fine for small-to-moderate files. - No extra setup or moving parts.

Disadvantages: - Memory usage scales linearly with file size, with no ceiling. - A single large file can exhaust available RAM and crash the entire n8n process, not just the one execution. - Works fine in testing and fails unpredictably in production, which makes it a nasty bug to catch in review.

Complexity: - Time: roughly O(n) in file size for parsing, same as any approach. - Space: O(n) in file size, held entirely in memory at once — this is the actual problem.

Optimal Solution

The fix isn't a code change — it's a configuration change that shifts where the bytes live while the workflow runs.

Set the environment variable:

```
N8N_DEFAULT_BINARY_DATA_MODE=filesystem
```

With this set, n8n stops attaching the full binary blob to the in-memory item. Instead, it writes binary data to disk and streams it as needed, passing around a reference instead of the raw bytes. The workflow logic — which nodes you use, what order they run in, what they do — doesn't change at all. Only the *storage mechanism* underneath changes.

Here's the flow with filesystem mode in place:

Step	Node	What happens
1	`Read Binary File`	Reads the file path, creates a binary property — now backed by disk, not RAM
2	`Extract From File`	Streams the binary data, parses rows into real JSON items you can loop, filter, and map
3	*(your logic)*	Transform the JSON exactly as before — this layer never touches binary data directly

Step	Node	What happens
4	`Convert to File`	Takes JSON and turns it back into a binary file
5	Write/Send node	Writes to disk, emails it, or uploads to storage — again, streamed rather than fully buffered

The shape of the pipeline is identical before and after the fix. What changed is invisible to the workflow logic and entirely about where the bytes physically sit while nodes are working on them.

One more piece worth understanding: Excel files (`.xlsx`) aren't plain text the way a CSV is. Under the hood, an XLSX file is a zipped bundle of XML. A generic text parser can't make sense of it, which is why n8n has a dedicated `Spreadsheet File` node that knows how to unpack that specific format. The output shape is the same as `Extract From File` though — binary goes in, JSON rows come out — so once you understand the binary-to-JSON pattern, it applies whether you're working with CSV, XLSX, or any other structured file format.

Python Code

n8n doesn't require you to write code for this fix, but if you're replicating the same binary-vs-streaming distinction in a Python pipeline (which is the exact same underlying problem), here's what "load fully into memory" versus "stream from disk" looks like in practice:

```
import csv
from pathlib import Path


def load_fully_in_memory(file_path: str) -> list[dict]:
    """Brute force: reads the entire file into memory at once."""
    with open(file_path, "r", newline="", encoding="utf-8") as f:
        # This materializes every row into memory simultaneously
        return list(csv.DictReader(f))


def stream_from_disk(file_path: str):
    """Optimal: yields rows one at a time, never holding the full file."""
    with open(file_path, "r", newline="", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            yield row


def process_large_csv(file_path: str, output_path: str) -> None:
    """Streams rows through a transform without ever buffering the whole file."""
    with open(output_path, "w", newline="", encoding="utf-8") as out:
        writer = None
        for row in stream_from_disk(file_path):
```

```
            row["processed"] = True
            if writer is None:
                writer = csv.DictWriter(out, fieldnames=row.keys())
                writer.writeheader()
            writer.writerow(row)
```

Code Walkthrough

- `load_fully_in_memory` uses `list(csv.DictReader(f))`, which forces every row to be read and materialized into a Python list before the function returns. This is the direct equivalent of n8n's default binary-in-memory mode — correct, but its memory footprint grows with file size.
- `stream_from_disk` is a generator (`yield` instead of `return`). It reads and hands off one row at a time, and the caller controls when the next row gets pulled. At any given moment, only a single row is resident in memory, regardless of whether the file has 500 rows or 50 million.
- `process_large_csv` ties it together: it reads from the stream and writes to disk row by row, never holding the full dataset in either direction. This mirrors exactly what filesystem mode does for n8n's binary property — read a chunk, process it, write it, move on.

Dry Run

Take a CSV with 3 rows:

```
name,score
Alice,90
Bob,75
Carol,88
```

With `stream_from_disk`: 1. Generator opens the file, creates a `DictReader`. 2. First `next()` call reads row 1: `{"name": "Alice", "score": "90"}` — yielded, memory holds only this row. 3. `process_large_csv` adds `"processed": True`, writes it to the output file immediately. 4. Second call reads row 2 (`Bob`), same treatment. Row 1 is already flushed to disk and out of memory. 5. Third call reads row 3 (`Carol`), same treatment. 6. Generator exhausts, loop ends.

At no point did all 3 rows exist in memory together — and this holds whether the file has 3 rows or 3 million. Compare that to `load_fully_in_memory`, where step 1 alone pulls every row into a list before anything else happens.

Complexity Analysis

- **Brute force (in-memory):** Time O(n), Space O(n) — n being total file size. Space is the failure point: it grows without bound as files grow, with no cap tied to available RAM.
- **Optimal (streaming/filesystem):** Time O(n) — same total work has to happen eventually — but Space is O(1) relative to file size (bounded by a small buffer/one row at a time), because you're never holding more than a constant-size slice of the data at once.

The time complexity is identical between the two approaches — you still have to read every byte eventually. The entire win is in space: trading "all of it, all at once" for "a little of it, continuously."

Alternative Solutions

A reasonable alternative to full filesystem mode is **chunked processing with explicit batching** — reading a file in fixed-size blocks (say, 1,000 rows at a time) and processing each batch before moving to the next, using something like pandas' `chunksize` parameter or manual pagination. This gives you more control over batch size and lets you parallelize batches, but it adds complexity you don't need if n8n's filesystem mode already solves the memory problem transparently. Use chunked batching when you need custom control over how data is grouped (e.g., batch-level transactions); use filesystem mode when you just need the memory problem to go away without touching your workflow logic.

Edge Cases

- **Empty file:** Both modes handle this fine — zero rows means zero memory regardless of storage mode, but your downstream nodes should still check for an empty JSON array before processing.
- **A single enormous row** (e.g., one record with a massive embedded blob): filesystem mode helps with overall file size, but an individual oversized record can still spike memory during the parse step — this is a different problem than aggregate file size.
- **Files with inconsistent encoding:** Streaming parsers can fail mid-file if encoding is inconsistent, whereas a fully-loaded parse might (incorrectly) succeed with garbled data. Validate encoding before processing either way.
- **Concurrent executions hitting disk simultaneously:** Filesystem mode means multiple workflow runs are now writing to the same disk location — make sure your n8n instance has enough disk space and I/O headroom for concurrent large-file workflows.
- **Max values — extremely large files:** Even with filesystem mode, individual nodes like `Extract From File` still need to parse content at some point; test your actual expected max file size, not just "bigger than before."

Common Mistakes

1. **Assuming the binary property is a file path reference** when it's actually the full byte content by default — this is the root misunderstanding behind the whole bug.
2. **Only testing with small sample files** and never validating against production-scale data before shipping a workflow.
3. **Setting the environment variable but not restarting n8n** — environment variable changes require a restart to take effect.
4. **Confusing this with a timeout issue** and increasing execution timeouts instead of addressing memory, which just delays the crash rather than fixing it.
5. **Not checking available disk space** after switching to filesystem mode — you've traded a memory constraint for a disk constraint, and large files still need somewhere to live.
6. **Trying to parse XLSX with a generic text reader** instead of the dedicated `Spreadsheet File` node, since XLSX is a zipped XML bundle, not plain text.
7. **Forgetting that binary data must be converted back with `Convert to File`** before writing, emailing, or uploading — JSON alone can't be written as a file.

Interview Questions

- What's the practical difference between loading a file fully into memory versus streaming it, and when does that difference actually matter?
- How would you detect that a service is failing due to memory pressure rather than a logic error?
- Design a system that processes arbitrarily large CSV files without requiring memory proportional to file size.
- What's the time and space complexity tradeoff (if any) between batch processing and full streaming?
- How do environment-variable-driven configuration changes differ from code changes in terms of deployment risk?

Similar Problems

- **LeetCode 146 (LRU Cache)** — similar in spirit: managing what stays "hot" in memory versus what gets evicted, under a fixed capacity constraint.
- **LeetCode 239 (Sliding Window Maximum)** — another case where holding a full dataset in memory is unnecessary; a bounded window is sufficient.
- **"External Sort" style problems** — classic computer science problems about processing data larger than available memory by processing it in chunks and merging results.
- **Streaming JSON parsers (ijson in Python)** — the direct code-level equivalent of what filesystem mode does for n8n's binary data.

Key Takeaways

The lesson isn't really about CSVs or even about n8n specifically. It's about a pattern that shows up in every system you'll ever build: anything that has to hold data fully in memory to operate on it will eventually hit a wall as that data grows, and the wall shows up exactly when you least expect it — in production, on real data, long after your tests passed. Learning to ask "where does this data actually live while the system is working on it?" is a transferable skill that will save you from this exact category of bug, in any language, in any pipeline, for the rest of your career.

Watch the Video

For the full walkthrough — including watching the crash happen live and the fix applied in real time — check out the video here: {LINK}

About the Series

This lesson is part of **Fun with Learning Technology**, a series built around teaching the real mechanics behind the tools developers use every day — not just "click this, then click that," but *why* it works the way it does, so you can diagnose problems you've never seen before instead of just memorizing steps.

Call To Action

If this saved you from a crash you haven't hit yet, do me a favor: like the video on YouTube, subscribe so you don't miss the next lesson, and subscribe to the Substack for the deeper written breakdowns that don't fit in a video. Drop a comment if you've hit this exact crash before — I want to hear how you found it. And if you know someone who's about to ship an n8n workflow to production, send this their way before they find out the hard way.

Quick Review

n8n Binary Data: when to think of it vs JSON fields?

Think binary when item holds file-like content (PDF, image, CSV blob) alongside JSON — two separate compartments per item, not one.

Where does binary data actually live during a workflow run?

Held in memory by default per item; large files can blow memory unless offloaded to filesystem mode.

Myth about the binary property field name?

It's not fixed to 'data' — you can name it anything; nodes just need to reference the correct property key.

Why/how do you 'flip binary to disk'?

Enable filesystem binary-data mode so large files write to disk instead of staying in memory, avoiding crashes on big files.

Extract from File (CSV) vs Spreadsheet File (XLSX) node?

CSV node parses plain-text delimited binary into JSON rows; Spreadsheet File node parses binary XLSX/ODS sheets into JSON rows.

Common bug when writing files back out in n8n?

Forgetting to convert JSON back into a binary property before writing — Write Binary File expects binary data, not raw JSON.
